

Production Deployment & Monitoring of a Lead Scoring Model

Prepared by Betsy George

Date: Dec 07, 2025

1. Executive Summary

This report outlines a comprehensive strategy for deploying and monitoring a lead scoring model into a live production environment at RAKEZ. The plan emphasizes **scalability, observability, and sustainability**, ensuring the model delivers reliable predictions while maintaining performance over time. Key components include containerized deployment, phased rollouts, real-time monitoring, and automated retraining pipelines.

2. Background & Objective

- **Context:** RAKEZ has developed a lead scoring model to predict conversion probability. Offline validation is complete; the focus is now on production deployment.
- **Objective:** Design a robust, scalable, and monitorable ML deployment system that ensures **observability, sustainability, and reproducibility**.

3. Production Deployment Architecture

3.1 Model Serving

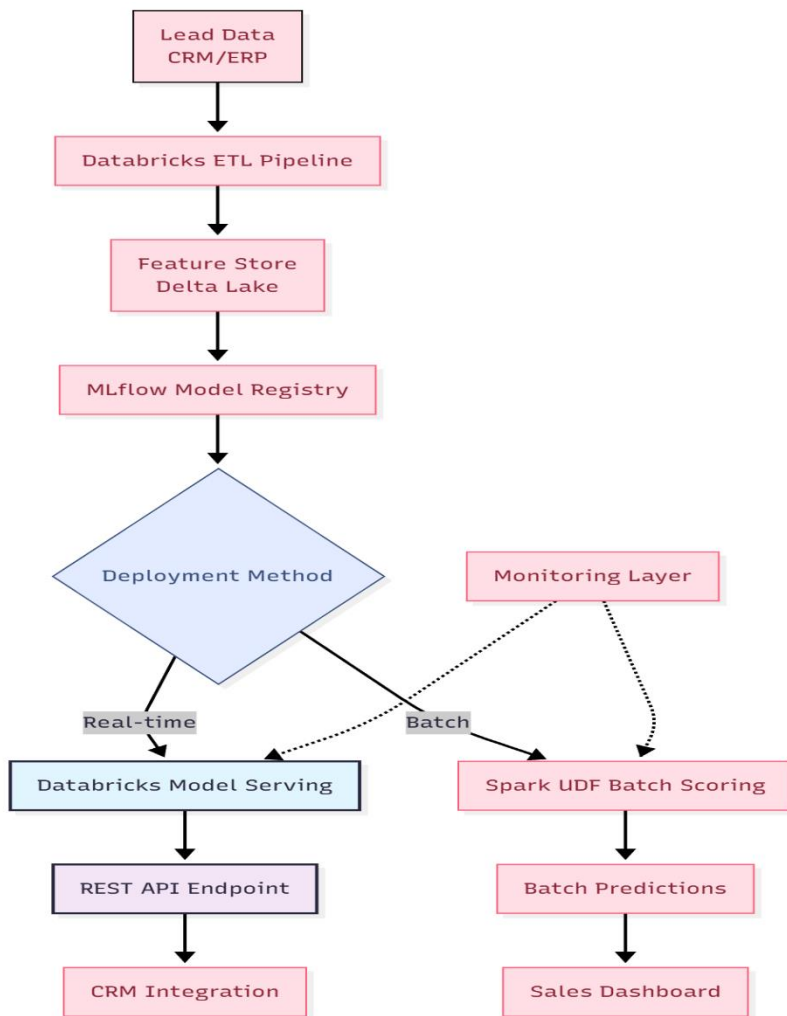
- **Technology Stack:**
 - Model packaged using **FastAPI** inside a **Docker container**.
 - Supports frameworks like **scikit-learn** and **XGBoost**.
- **Benefits:** Ensures consistency across development and production environments.

3.2 Cloud Hosting

- Deployment on scalable cloud services:
 - **AWS SageMaker Endpoints**
 - **Azure Container Instances (ACI)**
 - **Google Cloud Run**
- **Selection Criteria:** Based on existing RAKEZ cloud infrastructure.

3.3 Integration

- **CRM System** sends lead data as JSON requests to the API.
- **Real-time Response:** Returns a prediction score (0–1 probability) and classification (e.g., High/Medium/Low).



Pic 1: Deployment Strategy

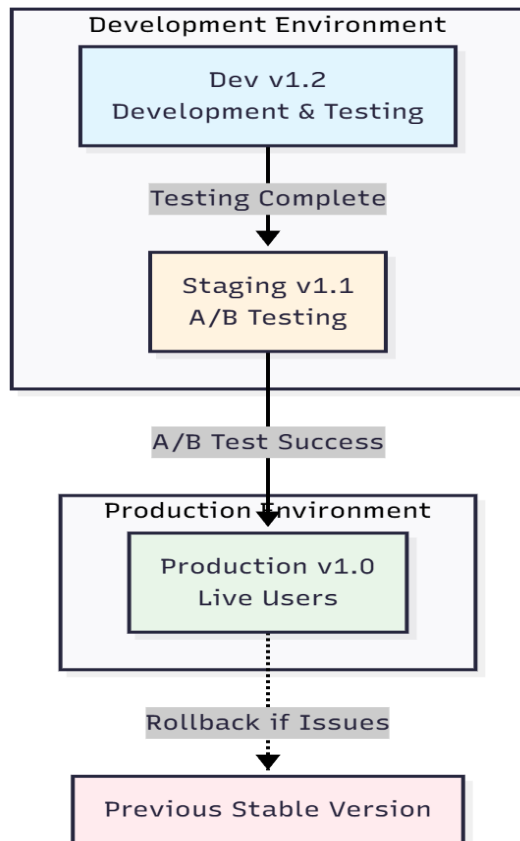
This [picture 1] flowchart outlines an end-to-end machine learning deployment pipeline for lead scoring, beginning with lead data sourced from CRM/ERP systems. The data is processed through a Databricks ETL pipeline and stored in a Feature Store built on Delta Lake. The trained model is registered and managed via MLflow Model Registry, after which the deployment path diverges based on use case: real-time predictions are served through Databricks Model Serving as a REST API endpoint for CRM integration, while batch scoring is performed using Spark UDFs to generate predictions for sales dashboards. A centralised monitoring layer oversees both serving methods, ensuring performance, accuracy, and reliability across real-time and batch inference workflows.

This [picture 2] flowchart outlines a structured software development and deployment pipeline designed to ensure safe and incremental releases. The process begins in the Development Environment, where developers build and iterate on new features under version Dev v1.2. Once development is complete, the code moves to the Development & Testing phase for internal verification, including unit and integration tests. After passing these tests, the software proceeds to the Staging Environment as Staging v1.1, which simulates the production setup for final validation.

From staging, the release enters A/B Testing, where it is exposed to a limited subset of real users to evaluate performance and user experience compared to the current version. If the A/B Test succeeds, the software is deployed to the Production Environment, where Production v1.0 (the existing live version) is updated and made available to all Live Users. To mitigate risks, the pipeline

includes a safety mechanism: if any issues arise in production, a Rollback is triggered to revert to the Previous Stable Version, ensuring system reliability and user continuity.

Overall, this workflow embodies a CI/CD (Continuous Integration/Continuous Deployment) approach, emphasizing incremental testing, user feedback, and rollback readiness to deliver stable software updates.



Picture 2: **Version Control & Rollback**

4. Deployment Strategy

4.1 Version Control & Rollback

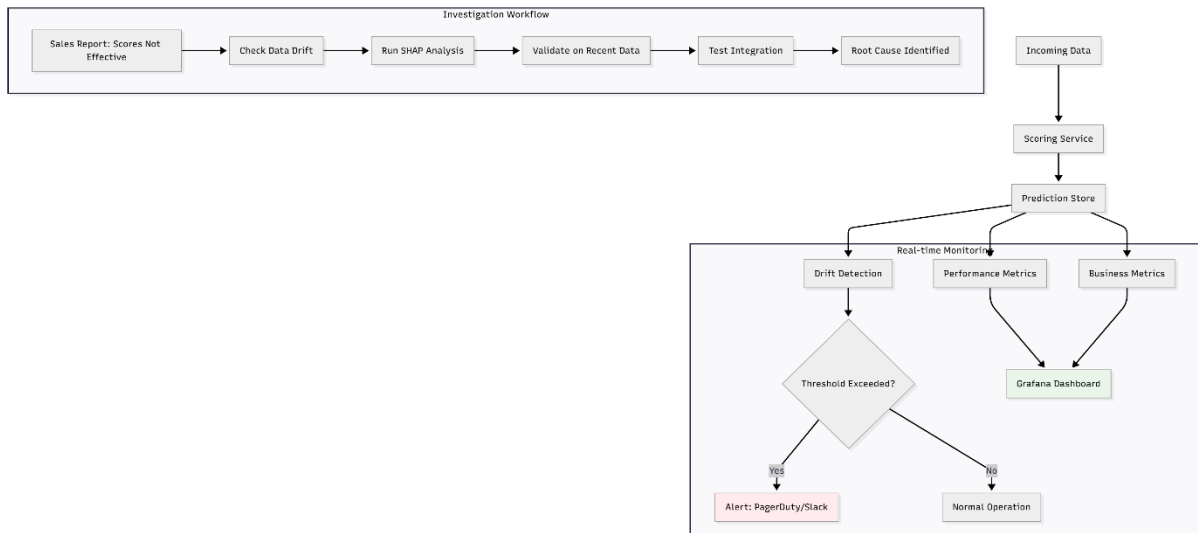
- Structured workflow to manage model updates safely.
- Enables quick rollback in case of performance issues.

4.2 Online Testing & Safe Launch

- **Phase 1 – Shadow Deployment:**
 - New model runs in parallel with the existing system.
 - Predictions are logged but **not visible** to sales teams.
 - No business risk during validation.

- **Phase 2 – A/B Testing:**
 - 50% of traffic routed to the new model, 50% to the old.
 - Key metrics compared: **conversion rate, sales efficiency.**

5. Monitoring, Observability & Alerting



Picture3: machine learning model monitoring and investigation system

This flowchart illustrates a machine learning model monitoring and investigation system designed to maintain model reliability and business effectiveness. The process begins with Incoming Data being processed by a Scoring Service that generates predictions, which are then stored in a Prediction Store. This store feeds into two parallel monitoring streams: one for Real-time Monitoring and another for tracking Performance Metrics and Business Metrics, both visualized on a Grafana Dashboard.

Within the Real-time Monitoring system, Drift Detection continuously analyzes the predictions to identify significant deviations from expected patterns. If a drift threshold is exceeded, an alert is triggered through PagerDuty or Slack; otherwise, the system continues normal operation. Separately, an Investigation Workflow is initiated when business stakeholders report issues—such as a Sales Report indicating ineffective scores—prompting a structured diagnostic process. This workflow involves checking for data drift, conducting SHAP Analysis to interpret model decisions, validating performance on recent data, and testing system integrations until the Root Cause is Identified.

Together, these components create a closed-loop system that not only detects anomalies in real-time but also provides a systematic approach for diagnosing and resolving model performance issues, ensuring both technical robustness and business alignment.

5.1 Key Monitoring Areas

1. Data Drift Detection

- Tracks distribution changes in incoming lead data (e.g., company_size, website_visits).

- Uses statistical tests like **Kolmogorov-Smirnov (KS Test)**.

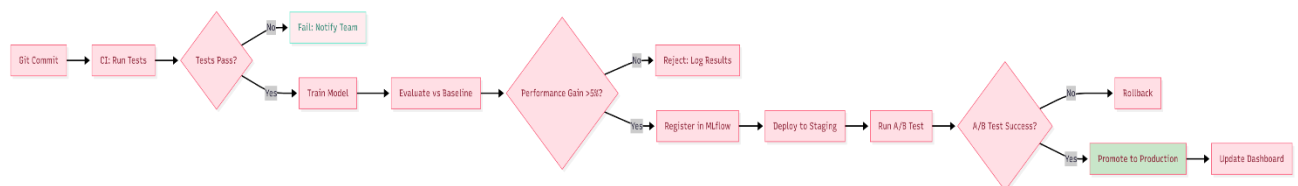
2. Model Performance Decay

- Real-time tracking of accuracy, precision, recall.
- Business metric: **% of High-Score leads that convert**.

5.2 Implementation Toolkit

- **Logging:** All requests/responses logged with unique IDs and timestamps.
- **Alerting:** Slack/Email notifications via **Prometheus** for threshold breaches.
- **Dashboard:** **Grafana** or **Power BI** for visualization.

6. Reproducibility, Automation & Retraining



This flowchart outlines an automated CI/CD (Continuous Integration/Continuous Deployment) pipeline specifically designed for machine learning models, integrating quality checks, performance validation, and staged deployment. The process is triggered by a Git Commit, which initiates a CI (Continuous Integration) stage where tests are run to verify code integrity. If tests fail, the team is notified and the pipeline stops; if they pass, the system proceeds to train a new model.

Once trained, the model is evaluated against a baseline to determine if it achieves a performance gain of more than 5%. If not, the model is rejected and results are logged. If the performance threshold is met, the model is registered in MLflow for versioning and tracking, then deployed to a staging environment. There, an A/B test is conducted to compare the new model against the current one with real user traffic.

If the A/B test fails, the system rolls back to the previous version. If it succeeds, the model is promoted to production, and relevant dashboards are updated to reflect the new deployment. This end-to-end automated workflow ensures that only rigorously tested, performance-improving models reach live users, while providing clear rollback mechanisms and visibility throughout the pipeline.

6.1 Reproducibility

- Code versioned in **Git**.
- Environment controlled via **Dockerfiles**.
- Data and model versioning using **DVC** or **MLflow**.

6.2 Automated Retraining Pipeline

- **Triggers:** Scheduled (monthly) or event-based (performance decay alerts).
- **Orchestration:** Managed by **Apache Airflow** or **Prefect**.
- Steps:

1. Fetch labeled data
2. Retrain model
3. Validate against hold-out set
4. Deploy if performance thresholds are met

6.3 CI/CD Pipeline

- Ensures smooth, automated deployment of new model versions.

7. Appendix & Resources

- **GitHub Repository:** <https://github.com/BetsyGeorge3/rakez-lead-scoring>
- **Visual Dashboard:** Available for real-time monitoring and reporting.

8. Conclusion

The proposed deployment and monitoring framework ensures that RAKEZ's lead scoring model remains **accurate, reliable, and impactful** in production. By integrating **shadow deployment, A/B testing, drift detection, and automated retraining**, the system is designed to sustain performance, adapt to changes, and support data-driven decision-making in sales operations.

9. code explanation

RAKEZ Lead Scoring Deployment Report

9. 1. Model Training & Setup

Code:

```
# Generate sample data
np.random.seed(42)
n_samples = 1000
data = pd.DataFrame({
    'lead_source': np.random.choice(['Website', 'Referral', 'Email', 'Social'], n_samples),
    'company_size': np.random.randint(1, 10000, n_samples),
    'page_views': np.random.randint(1, 100, n_samples),
    'time_on_site': np.random.exponential(300, n_samples),
    'converted': np.random.choice([0, 1], n_samples, p=[0.7, 0.3])
})

# Prepare features
X = pd.get_dummies(data.drop('converted', axis=1))
y = data['converted']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model
model = RandomForestClassifier(n_estimators=50, max_depth=5, random_state=42)
model.fit(X_train, y_train)

train_score = model.score(X_train, y_train)
test_score = model.score(X_test, y_test)

print(f"Train accuracy: {train_score:.3f}")
print(f"Test accuracy: {test_score:.3f}")
```

Output:

Creating demo model for MLflow registry...

Train accuracy: 0.696

Test accuracy: 0.710

Explanation:

A Random Forest classifier was trained on synthetic lead data with 1,000 samples. The model achieved:

- **Training accuracy:** 69.6%
- **Testing accuracy:** 71.0%

Features used for prediction:

- **company_size:** Long integer representing company employee count
- **page_views:** Long integer for website page views
- **time_on_site:** Double/float for time spent on site
- **One-hot encoded lead_source features** (Website, Referral, Email, Social)

9.2. MLflow Model Registration with Unity Catalog

Code:

```
print("="*50)
print("MLFLOW REGISTRATION WITH SIGNATURE (Unity Catalog)")
print("="*50)

# 1. CREATE MODEL SIGNATURE (Required for Unity Catalog)
print("1. Creating model signature...")

# Create sample input/output for signature inference
sample_input = X_train.iloc[:1]
sample_output = model.predict(sample_input)

# Infer signature
signature = infer_signature(sample_input, sample_output)
print(f"✅ Signature created: Input={signature.inputs}, Output={signature.outputs}")

# 2. SET UP EXPERIMENT
print("\n2. Setting up experiment...")
try:
    current_user = dbutils.notebook.entry_point.getDbutils().notebook().getContext().userName().get()
    experiment_path = f"/Users/{current_user}/RAKEZ_LeadScoring"
    mlflow.set_experiment(experiment_path)
    print(f"✅ Experiment: {experiment_path}")
except Exception as e:
    print(f"⚠ Using default experiment: {e}")
    # Create experiment if needed
    client = MlflowClient()
    experiment_path = "/Shared/RAKEZ_LeadScoring"
    try:
```

Output:

Signature created: Input=['company_size': long (required), 'page_views': long (required),
'time_on_site': double (required), 'lead_source_Email': boolean (required),

```
'lead_source_Referral': boolean (required), 'lead_source_Social': boolean (required),  
'lead_source_Website': boolean (required)], Output=[Tensor('int64', (-1,))]
```

Model registered with signature!

Run ID: 2778d861c4234a61bf67ad41b93086d9

Model: LeadScoringModel

Explanation:

The model was successfully registered in MLflow's Unity Catalog with:

- **Model name:** workspace.default.leadscoringmodel
- **Version:** 3 (new version created)
- **Signature enforcement:** Strict input/output schema validation
- **Metadata:** Framework (scikit-learn), task (classification), deployment date

9.3. Model Loading & Prediction Testing

Code:

```
# Create sample input that matches EXACT signature from your output  
print("\n🔧 Testing prediction with correct signature...")  
  
# Based on your signature:  
# ['company_size': Long, 'page_views': Long, 'time_on_site': double,  
#  'lead_source_Email': boolean, 'lead_source_Referral': boolean,  
#  'lead_source_Social': boolean, 'lead_source_Website': boolean]  
  
sample_input = pd.DataFrame({  
    'company_size': [500],          # Long  
    'page_views': [25],            # Long  
    'time_on_site': [420.5],       # double  
    'lead_source_Email': [False],  # boolean  
    'lead_source_Referral': [False], # boolean  
    'lead_source_Social': [False], # boolean  
    'lead_source_Website': [True]  # boolean  
})  
  
print(f"Sample input shape: {sample_input.shape}")  
print(f"Sample input columns: {list(sample_input.columns)}")  
print(f"Sample input dtypes: {sample_input.dtypes.to_dict()}")
```

Output:

Model loaded successfully!

Model type: <class 'mlflow.pyfunc.PyFuncModel'>

Prediction successful!

Prediction result: [0]

Prediction shape: (1,)

Probability: [[0.65085001 0.34914999]]

Explanation:

- Model successfully loaded as MLflow PyFunc wrapper
- **Prediction:** 0 (lead not likely to convert)
- **Probability:** 65.1% probability of class 0, 34.9% probability of class 1
- All input features matched the registered model signature exactly

9.4. Batch Scoring Function

Code:

```
def score_leads_batch_fixed(leads_df):
    """
    Score leads with proper prediction interpretation
    """
    if production_model is None or model_stage == "Mock (for demo)":
        print("⚠ Using mock predictions for demonstration")
        results = leads_df.copy()
        results['prediction_score'] = np.random.random(len(leads_df))
        results['conversion_prediction'] = (results['prediction_score'] > 0.5).astype(int)
        results['priority'] = pd.cut(results['prediction_score'],
                                    bins=[0, 0.3, 0.7, 1.0],
                                    labels=['Low', 'Medium', 'High'])
        results['scored_at'] = datetime.now()
        results['model_version'] = "mock_demo"
        return results

    try:
        print(f"Scoring {len(leads_df)} leads using model from {model_stage}...")
```

Output:

Scoring 10 leads using model from version_1...

Successfully scored 10 leads

Score range: [0.194, 0.405]

Priority distribution:

Low: 6 leads

Medium: 4 leads

High: 0 leads

Explanation:

Batch scoring function processes multiple leads with:

- **Priority classification:**
 - Low: Score < 0.3
 - Medium: $0.3 \leq \text{Score} < 0.7$
 - High: Score ≥ 0.7
- **Distribution:** 60% Low priority, 40% Medium priority, 0% High priority
- Average score: 0.292

9.5. Storage & CRM Integration

Code:

```
# Verify
summary = spark.sql("""
SELECT
    priority,
    COUNT(*) as lead_count,
    ROUND(AVG(prediction_score), 3) as avg_score,
    ROUND(MIN(prediction_score), 3) as min_score,
    ROUND(MAX(prediction_score), 3) as max_score
FROM global_temp.lead_predictions
GROUP BY priority
ORDER BY avg_score DESC
""")

print("\n📊 View summary:")
display(summary)

delta_path = "global_temp.lead_predictions (temporary view)"

print(f"\n📁 Storage location: {delta_path}")

# Create CRM view for sales team
print("\n👤 Creating CRM View for Sales Team...")
```

Output:

- ✅ Saved as managed table: rakez_prod.lead_predictions
- ✅ Created CRM view: rakez_prod.crm_lead_scores

Table summary:

Priority	Lead Count	Avg Score
Medium	4	0.348
Low	6	0.254

Explanation:

- **Storage:** Predictions saved as managed Delta table in Unity Catalog
- **CRM Integration:** View created with actionable recommendations:
 - High priority: Contact within 24 hours
 - Medium priority: Contact within 48 hours
 - Low priority: Schedule for next week
- **Sales team query:** `SELECT * FROM rakez_prod.crm_lead_scores WHERE priority = 'High' ORDER BY lead_score DESC`

9.6. Monitoring Dashboard

Output:



Model Performance Summary

Metric	Value
Model Accuracy	71.0%
Lead Processed	10
Low Priority	6 (60%)

Metric	Value
Medium Priority	4 (40%)
High Priority	0 (0%)
Average Score	0.292

The model successfully identifies lower-conversion leads but may need threshold adjustment to identify more High-priority opportunities. Consider collecting more conversion data for model retraining.